

Crea una presentación académica, profesional y clara titulada:

“De componentes reactivos a APIs REST: integrando Vue 3 con Node y Express”

La audiencia son estudiantes que conocen HTML, CSS y JavaScript básico, pero están aprendiendo programación asincrónica, consumo de servicios y arquitectura frontend/backend. Usa un lenguaje sencillo, ejemplos progresivos y explicaciones visuales. La presentación debe relacionarse con un ERP contable construido con Vue 3, Vite, Vuetify y Vue Router.

Objetivo general

Explicar cómo transformar una aplicación Vue que utiliza datos locales en una aplicación que consume APIs RESTful construidas con Node.js y Express.

Objetivos de aprendizaje

Al finalizar, los estudiantes podrán:

1. Explicar qué es una API y diferenciar frontend de backend.
2. Identificar recursos, endpoints y métodos HTTP.
3. Interpretar los códigos de estado más comunes.
4. Comprender JSON como formato de intercambio.
5. Explicar promesas, `async`, `await`, `try/catch` y `Promise.all`.
6. Consumir una API con `fetch` y reconocer cuándo usar Axios.
7. Relacionar reactividad de Vue con datos recibidos desde un servidor.
8. Probar endpoints con Postman o Insomnia.

Estructura obligatoria

Genera entre 14 y 16 diapositivas. Cada diapositiva debe tener:

- Un título breve.
- Máximo cinco ideas principales.
- Un ejemplo concreto del ERP.
- Una sugerencia visual o diagrama.
- Notas del expositor con una explicación de 60 a 90 segundos.

Diapositivas

1. Portada

- Título, curso, estudiante y fecha.
- Imagen o diagrama sencillo de Vue conectado con Express.

2. El problema inicial

- La interfaz usa datos escritos directamente en los componentes.
- Limitaciones de los datos locales.
- Necesidad de separar interfaz y datos.

3. Arquitectura frontend/backend

- Vue en `localhost:5173`.
- Express en `localhost:3000`.
- Flujo solicitud-respuesta.
- Incluir un diagrama de flechas.

4. ¿Qué es una API?

- Definición con una analogía sencilla.
- API como contrato entre aplicaciones.
- Ejemplo: Vue solicita contactos a Express.

5. API RESTful

- Recursos y URLs.
- Endpoints del ERP: `/api/contactos` y `/api/movimientos`.
- Diferencia entre recurso y acción.

6. Métodos HTTP

- `GET`, `POST`, `PUT`, `DELETE`.
- Relación con consultar, crear, actualizar y eliminar.
- Tabla pequeña con ejemplos del ERP.

7. JSON y códigos de estado

- Ejemplo de objeto JSON.
- `200`, `201`, `204`, `400`, `404`, `500`.
- Explicar que el código comunica el resultado.

8. Servidor Express mínimo

- `express()`.
- `express.json()`.
- `cors()`.
- Una ruta `GET`.
- Mostrar un fragmento corto, no un archivo completo.

9. Programación asíncrona ES6+

- Por qué una solicitud HTTP tarda.
- Promesas y sus estados.
- Diferencia entre código síncrono y asíncrono.
- Evitar la idea incorrecta de que `await` bloquea todo el navegador.

10. Async/await y manejo de errores

- Mostrar un ejemplo breve con `try/catch/finally`.
- Explicar `respuesta.ok` en Fetch.
- Estados de carga y error.

11. Consumo con Fetch

- Ejemplo de **GET**.
- Ejemplo de **POST** con **JSON.stringify**.
- Explicar headers y body.

12. Consumo con Axios

- Ejemplo de instancia con **baseUrl**.
- Comparar brevemente Fetch y Axios.
- Recomendar no mezclar ambas herramientas sin un criterio claro.

13. Componentes y reactividad en Vue

- **ref**, **computed**, **onMounted**.
- Props y eventos.
- Cómo una respuesta de la API actualiza la vista.
- Ejemplo con **contactos.value = datos**.

14. Prueba de endpoints

- Flujo en Postman o Insomnia.
- Solicitud **GET /api/contactos**.
- Solicitud **POST /api/contactos**.
- Revisar URL, método, body, código y respuesta.

15. Caso completo del ERP

- El usuario registra un movimiento.
- Vue envía **POST**.
- Express valida y responde **201**.
- Vue actualiza la lista y los totales mediante reactividad.
- Incluir un diagrama secuencial.

16. Cierre y actividad

- Resumir los conceptos.
- Proponer una actividad: implementar contactos y movimientos.
- Incluir tres preguntas de comprobación:
 1. ¿Qué diferencia hay entre **POST** y **PUT**?
 2. ¿Por qué se debe comprobar **respuesta.ok**?
 3. ¿Cómo sabe Vue que debe actualizar la tabla?

Requisitos técnicos y visuales

- Usa un diseño sobrio, moderno y académico.
- Utiliza una paleta con fondo claro, azul petróleo, verde para éxito y rojo para errores.
- Usa tipografía legible y alto contraste.
- No llenes las diapositivas con párrafos largos.
- Usa diagramas de arquitectura, tablas pequeñas y fragmentos de código breves.
- Conserva los nombres reales: **erp-contable**, **erp-api**, **/api/contactos** y **/api/movimientos**.
- Todos los ejemplos deben usar JavaScript, no TypeScript.

- No inventes librerías diferentes a Vue, Vite, Vuetify, Express, Fetch, Axios, Postman e Insomnia.
- Explica cada acrónimo la primera vez que aparezca.
- Incluye notas del expositor separadas del contenido visible.
- No afirmes que los datos persisten: aclara que inicialmente están en memoria y se pierden al reiniciar Express.
- Finaliza con una diapositiva de preguntas y una actividad práctica.

Formato de salida

Entrega:

1. El contenido completo de cada diapositiva.
2. Las notas del expositor para cada diapositiva.
3. Una descripción visual de cada diagrama.
4. Un guion final de máximo 12 minutos.
5. Una rúbrica breve con cuatro criterios: conceptos, demostración técnica, claridad y manejo de preguntas.